

在 IaaS 云中托管工作流即服务的费用优化

Amelie Chi Zhou, Bingsheng He and Cheng Liu
Nanyang Technological University

摘要—近年来, 科学应用及其他数据密集型应用的工作流不断进入基础设施即服务 (IaaS) 云, 许多工作流服务提供商开始提供工作流即服务 (WaaS)。WaaS 提供商的主要关切, 是尽量降低在 IaaS 云中执行工作流的费用。尽管已有研究关注这一问题, 其中大多数仍假设任务执行时间和定价方案是静态的, 并将满足确定性截止期作为服务质量 (QoS) 目标。然而, 云环境具有动态性: 并发执行之间的干扰会造成性能变化, Amazon EC2 竞价实例等服务则会造成价格变化。因此, 我们认为, WaaS 提供商应当为 IaaS 云中的各个工作流提供概率性能保证。本文提出名为 *Dyna* 的概率调度框架, 在提供概率截止期保证的同时尽量降低费用。框架包括一种针对性能动态性的、基于 A^* 的实例配置方法, 以及一种利用竞价实例的混合实例配置细化方法。我们在 Amazon EC2 上以三个真实科学工作流应用开展实验。结果表明: (1) 该框架能够准确满足用户要求的概率截止期保证; (2) 与已有方法相比, 该框架能够有效降低费用。

关键词—云计算, 云环境动态性, 竞价价格, 费用优化, 科学工作流。

1 引言

云计算已成为各种应用广泛采用的计算基础设施。云计算的一项吸引力在于按使用量付费: 用户只需为实际使用的存储量和计算时长付费。借助这一特性, 用户无需实际拥有一套云设施, 也能进行大规模计算。近年来, 各类科学应用和数据密集型应用的许多工作流被部署并托管在基础设施即服务 (IaaS) 云上, 例如 Amazon EC2 及其他云服务提供商 [1], [2]。在这些应用中, 工作流提交到云中执行, 通常还会附带一个用于满足 QoS 要求的截止期 [3], [4]。这形成了一种在云中托管工作流的软件即服务模式, 我们称之为工作流即服务 (WaaS)。WaaS 提供商根据用户工作流的执行情况及其 QoS 要求收取费用; 与此同时, 他们需要从 IaaS 云租用资源, 因而产生费用。对云中的 WaaS 提供商而言, 费用与性能通常是最重要的两个优化因素。本文研究 WaaS 提供商能否、以及如何为各个工作流提供性能保证的同时, 降低托管 WaaS 的费用。

费用优化一直是网格计算和云计算环境中的经典研究课题。在网格计算时代, 研究者已广泛研究成本感知优化技术, 并处理各种问题: 在给定性能要求下最小化成本 [5], 在给定预算下最大化性能 [6], 以及同时满足成本与性能约束的调度优化 [7]。进入云计算时代后, 按使用量计费、虚拟化和弹性等特性带来了新的挑战与机会 [3], [8]。例如, 大多数云提供商采用实例小时计费模式, 不足一小时的使用量也会向上取整为一小时。虽然已出现其他计费模式, 例如 Google 的 IaaS 服务按使用分钟数计费, 但按小时计费仍是最常用的模式。近年来, 许多研究针对云计算的这些特性, 通过资源分配与任务调度优化费用, 例如 [3], [4], [8], [9], [10], [11], [12]。尽管这些研究已展示降低费用的效果, 但它们都假设任务执行时间是静态的, 并且只考虑固定定价方案, 即 Amazon 术语中的按需实例。具体而言, 它们存在以下局限。

首先, 云从设计上就是共享基础设施。云中的计算、存储、网络等资源, 会被许多并发作业或任务共享。已有研究 [13], [14] 已经表明, I/O 和网络性能存在显著波动。因此, 以往研究所采用的静态任务执行时间假设, 例如 [3], [4], [8], [9], [10], [11], [12] 中的假设, 在云中并不成立。在静态执行时间假设下, 现有成本优化方法和算法试图满足传统 QoS 概念, 即以 100% 的保证满足软截止期。下文将这一传统概念称为“确定性截止期”。然而, 由于性能变化, 一个作业在优化后的实际执行时间可能以不同概率取不同数值。要求所有可能的执行时间都满足确定性截止期, 既昂贵又缺乏意义。因此, 确定性

截止期并不适合用于在动态云环境中提供性能保证, 已有研究需要重新审视并适应这种性能变化。

其次, 云已发展为一种经济市场 [15], 其定价也具有动态性。Amazon EC2 提供竞价实例, 价格由市场供需决定。以往研究 [3], [5], [16], [17], [4], [18] 只考虑固定定价方案; 当竞价实例存在时, 需要重新审视这些结果。另一方面, 竞价实例可用于降低费用 [19], [20], [21], [22], [23], [24], 因为其价格通常远低于同类型按需实例的价格。但是, 当出价低于竞价价格时, 竞价实例可能随时被终止, 即发生出价失效事件。使用竞价实例可能因故障而产生过长延迟。多数已有研究没有考虑各个工作流的截止期约束。

这两种云动态性使得 WaaS 提供商在满足各个工作流 QoS 要求的同时最小化费用, 变得更具挑战。它们给问题增加了两个新维度, 并大幅扩大了解空间, 详见第 3 节。此外, 确定性截止期这一概念会使优化依赖最坏情况下的性能或价格预测。更糟的是, 某些情形下最坏情况可能未知或无法预测, 例如 Amazon EC2 的历史竞价价格中存在一些异常高价。

为处理性能和价格的动态性, 我们以概率性能保证来表示 QoS。每个工作流对应一个 $p\%$ 的概率截止期保证。确定性截止期保证可以视为概率保证为 100% 的特例。WaaS 提供商保证, 在动态云环境中, 工作流执行时间满足其分布的第 p 百分位要求。这类似于许多 IaaS 云提供商给出 99.95% 的资源可用性保证 [25]。基于这一概念, 我们提出名为 *Dyna* 的概率框架, 在满足用户预先规定的各个工作流概率性能保证的同时, 尽量降低 WaaS 提供商的费用。框架包含一系列专门针对云动态性的静态与动态费用优化。我们构建概率模型, 描述 I/O 性能、网络性能以及竞价价格的动态性 [26], [27]。进一步地, 我们提出同时使用竞价实例和按需实例的混合执行方式: 用竞价实例争取降低费用, 以按需实例作为满足截止期约束的最后保障。

我们从真实云提供商 Amazon EC2 获取校准数据, 为 I/O 与网络性能以及竞价价格建立概率模型, 并以三个真实工作流应用在真实云环境中开展实验。实验主要得到以下两个结果。

- 1) 经过 Amazon EC2 数据校准, *Dyna* 能够准确描述云动态性, 其优化结果对应的概率总能达到、或略高于用户要求的概率性能保证。
- 2) 相比仅采用按需实例的其他先进算法, 混合实例配置方法可将费用显著降低 15–73%。

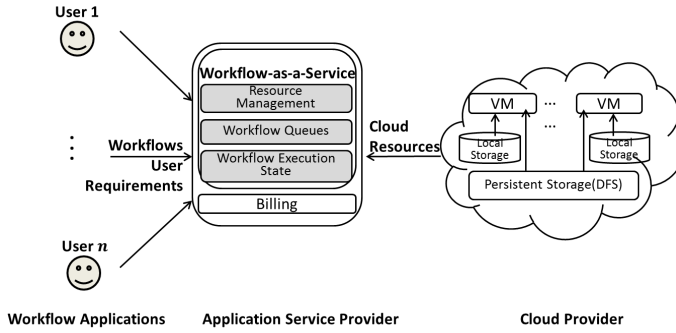


图 1. 本文的应用场景。图内文字译注：User 1、User n 为用户 1、用户 n；Workflows 为工作流，User Requirements 为用户需求；Workflow-as-a-Service 为工作流即服务；Resource Management 为资源管理，Workflow Queues 为工作流队列，Workflow Execution State 为工作流执行状态，Billing 为计费；Cloud Resources 为云资源；VM 为虚拟机，Local Storage 为本地存储，Persistent Storage (DFS) 为持久存储（分布式文件系统）；底部三栏依次为工作流应用、应用服务提供商和云提供商。

据我们所知，本文是首个面向云中 WaaS 提供商的概率调度框架。

本文其余部分安排如下：第 2 节给出问题形式化并回顾相关工作；第 3 节详细介绍框架设计；第 4 节报告实验结果；第 5 节总结全文。

2 背景与相关工作

本节先介绍应用场景与术语，再回顾相关工作。

2.1 应用场景

图 1 展示了本文的应用场景。我们考虑在 IaaS 云中以软件即服务模式提供工作流这一典型场景 [3], [1]，并将其称为工作流即服务 (WaaS)。在这种托管模式下，不同应用所有者提交若干带有不同参数的工作流类。工作流类表示结构相同、输入数据参数不同的一类工作流模板。WaaS 提供商允许用户为某个工作流类指定输入数据，从而实例化一次工作流执行，同时指定用于满足 QoS 要求的截止期。WaaS 提供商根据工作流类及其 QoS 要求收费；工作流类反映工作流的复杂度。在云中，可以看到许多结构和用途各不相同的应用 [1], [2]。用户，例如科学家和政府工作人员，可以提交用于预测的仿真任务或进行敏感性分析，也可以使用数据挖掘或机器学习技术分析科学数据。

采用不同的工作流调度与资源供应算法，会使 WaaS 提供商在 IaaS 云中运行服务的费用产生显著差异。考虑到云环境的动态性，本文的目标是为 WaaS 提供商构建一个概率调度框架，在满足用户 QoS 要求的同时尽量降低费用。

2.2 术语

实例。实例是云提供商提供的虚拟机。同类型实例具有相同数量的资源，例如 CPU 和 RAM，也具有相同的能力，例如 CPU 速度、I/O 速度和网络带宽。

获取实例需要不可忽略的获取时间。为简化分析，我们将该时间假设为常数 lag 。

实例可以是按需实例或竞价实例。本文采用 Amazon EC2 的实例定义。Amazon 使用按小时计费模式，实例使用时间中不足一小时的部分向上取整为一小时。用户不再需要实例时，可以终止按需实例或竞价实例。若由用户终止实例，不足一小时的使用时间仍需按一小时付费。对于竞价实例，如果实例因出价失效事件被终止，则不足一小时的使用部分无需付费。

表 1 给出了 2013 年 8 月 Amazon 美国东部区域四种竞价实例历史价格的统计信息，同时列出了对应按需实例的价格。

表 1
Amazon EC2 竞价价格统计 (美元/小时, 2013 年 8 月, 美国东部区域) 及按需价格。

实例类型	均值	<i>stdev</i>	最小值	最大值	按需价格
m1.small	0.048	0.438	0.007	10	0.06
m1.medium	0.246	1.31	0.0001	10	0.12
m1.large	0.069	0.770	0.026	40	0.24
m1.xlarge	0.413	2.22	0.052	20	0.48

可以得到两点观察：(a) 竞价实例通常比按需实例便宜，但也存在一些“异常值”，其最高价格远高于按需价格；(b) 不同实例类型的价格波动程度不同。这些观察与已有研究 [27], [26] 一致。

作业。作业表示为一组带有先后依赖约束的任务构成的工作流。作业具有截止期，该截止期是“软截止期”，区别于硬实时系统中的“硬截止期”。本文将作业截止期视为概率要求。假设一个工作流被规定了 $p\%$ 的概率截止期保证，WaaS 提供商保证，在动态云环境中，该工作流的执行时间满足其执行时间分布的第 p 百分位要求。

任务。根据总执行时间中占主导的部分，任务可分为计算密集型、I/O 密集型等不同类别。任务执行时间或响应时间，通常利用任务剖析、机器基准测试和统计分析等估计方法得到。本文采用简单的性能估计模型预测任务执行时间。工作流通常具有规则性与可预测性 [3], [5]，因此这一简单方法在实践中已足够准确，真实云环境中的实验也验证了这一点。具体而言，我们利用如下剖析信息估计任务在不同类型实例上的执行时间： $\langle \#instr, d_{seqIO}, d_{rndIO}, netInput, netOutput \rangle$ 。其中， $\#instr$ 表示任务需要执行的指令总数； d_{seqIO} 和 d_{rndIO} 分别表示对本地磁盘进行顺序访问与随机访问的 I/O 数据量； $netInput$ 和 $netOutput$ 分别表示需要读取与发送的输入、输出数据量。任务执行时间估计为 CPU 时间、I/O 时间和网络时间之和。

在估计过程中，CPU 时间由任务的 $\#instr$ 以及执行该任务的实例 CPU 频率决定。类似地，I/O 时间和网络时间分别由 I/O 数据量与网络数据量除以相应的 I/O 带宽和网络带宽得到。我们将云中的 I/O 与网络性能建模为概率分布，并据此估计任务在云中的动态执行时间。

实例配置。任务的混合实例配置定义为一个 n 维向量： $\langle (type_1, price_1, isSpot_1), (type_2, price_2, isSpot_2), \dots, (type_n, price_n, isSpot_n) \rangle$ 。其含义是，任务可能依次在 n 个属于 $type_i$ ($1 \leq i \leq n$) 类型的实例上执行。 $isSpot_i$ 指明实例是竞价实例还是按需实例。若实例 i 是竞价实例，则 $price_i$ 是指定出价；否则是按需价格。在本文的混合实例配置中，仅最后一维为按需实例，前面的各维均为竞价实例。这是因为按需实例能以 100% 的概率保证成功执行。

在竞价实例与按需实例的混合执行中，任务首先分配至其配置第一维所指示类型的竞价实例上，如果配置中存在该维。若任务在此竞价实例上失败，则重新分配到配置下一维所指示类型的实例，直至成功完成。由于最后一维对应按需实例，即使任务在此前所有竞价实例上均失败，也总能完成执行。

2.3 相关工作

许多工作与本文相关，这里重点讨论费用优化和云性能动态性方面最相关的研究。据我们所知，本文是首个同时描述云中性能动态性与价格动态性的调度框架。

成本感知优化。云计算按使用量付费的特性，吸引了大量关于动态资源供应的研究。动态虚拟机供应可以依据控制理论 [28], [29]、机器学习 [30] 以及模型 [31] 来决定。另一方面，带有截止期和预算约束的工作流调度也已被广泛研究，例如 [5], [16], [17], [4], [18], [32], [33]。Yu 等 [5] 为一个作业内的任务分配截止期，并使用遗传算法寻找最优调度计划。Killapi

等 [4] 研究费用与性能之间的权衡, 将其建模为数据库中的天际线操作。这些研究仅考虑单个工作流, 且仅使用按需实例。Malawski 等 [32] 为工作流集合提出动态调度策略。已有研究 [3], [34], [35] 基于各个任务的静态执行时间提出自动伸缩技术。与它们相比, Dyna 的独特之处在于, 将概率性能保证作为 QoS 目标, 而非确定性截止期。Dyna 通过显式描述云中的性能动态性, 尤其是 I/O 与网络性能变化, 来调度工作流。Buyya 等 [33] 提出采用任务复制的算法, 以提高满足截止期的可能性。

由于能够降低费用, Amazon EC2 竞价实例近年来受到广泛关注。相关工作大体可分为两类: 竞价价格建模 [26], [27], 以及利用竞价实例 [19], [20], [21], [22], [23], [24], [36]。

在竞价价格建模方面, Yehuda 等 [26] 对竞价价格进行逆向分析, 得到了与已有价格轨迹一致的模型。Javadi 等 [27], [37] 为不同竞价实例类型建立统计模型。这些模型均可用于本文的混合执行方式。

在利用竞价实例方面, Chohan 等 [21] 提出利用竞价实例加速 MapReduce 任务, 并指出在竞价实例上运行 MapReduce 作业时, 容错机制不可缺少。Yi 等 [19] 引入一些检查点机制, 降低竞价实例的成本。Ostermann 等 [36] 在网络资源不足时, 利用竞价实例执行大型工作流。进一步的研究 [23], [24] 结合不同出价策略使用竞价实例, 并引入检查点、任务复制和迁移等容错技术。这些研究仅使用竞价实例, 未像 Dyna 一样提供满足工作流截止期的保证。

云性能动态性。一些研究从不同方面评估云服务的性能 [38], [13], [14]。本文在 Amazon EC2 上的校准结果与 Schad 等 [13] 的研究一致。Iosup 等 [14] 针对科学计算负载, 对四个商业云提供商进行跨平台比较: Amazon EC2、GoGrid、ElasticHosts 和 Mosso。还有一些研究更深入地分析特定组件, 例如网络性能 [39], [40], 以及 I/O 干扰 [41], [42]。

已有一些方案试图减少云中的性能干扰与不可预测性, 例如改善网络性能 [43] 和 I/O 性能 [44], [45], [46]。然而, 按照云计算的设计, 云会被许多并发执行共享。因此, 资源干扰引起的性能动态性无法避免。本文以概率概念描述性能与成本的动态性, 进一步构建一个考虑这些动态性的概率调度框架, 以最小化费用。

3 框架设计与实现

我们先概述 Dyna 框架, 再讨论其所采用优化技术的设计细节。

3.1 框架概述

设计用于最小化 WaaS 提供商费用的框架时, 我们考虑以下原则。

- **有效性。**所提框架应在优化费用时特别考虑云环境中的性能与价格动态性。此外, 在满足各个工作流概率截止期保证的同时, 框架应能找到可与最优解相比、或接近最优解的方案。
- **通用性。**构建通用框架有两个原因。首先, 已有一些面向费用与性能的经典作业或任务调度优化 [5], [6], [7]; 使用框架可将这些现有优化整体整合起来。其次, 不同云提供商在性能与价格动态性方面, 可能具有不同的行为和服务。通用框架使我们能更灵活地为不同云提供商实现专门算法, 而不影响其他关键优化组件。
- **低运行时开销。**成本优化是一个在线过程, 应当足够轻量。需要在费用优化的质量与优化过程本身的运行时开销之间取得良好平衡。由于空间庞大, 彻底探索整个优化空间并不现实。

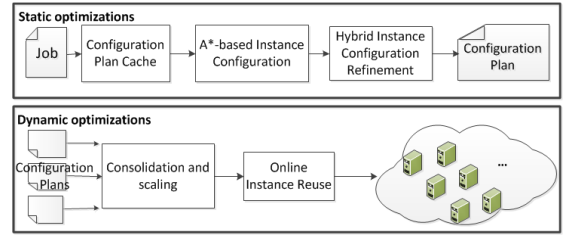


图 2. Dyna 框架中的静态与动态优化图内文字译注: 上半部为静态优化, 依次是作业、配置计划缓存、基于 A^* 的实例配置、混合实例配置细化和配置计划; 下半部为动态优化, 依次是配置计划、整合与伸缩、在线实例复用。

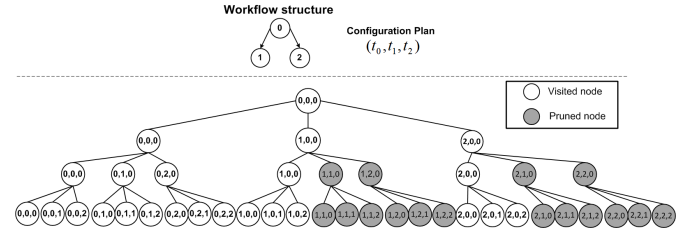


图 3. 本文 A^* 算法中配置计划搜索树的一个示例。图内文字译注: Workflow structure 为工作流结构, Configuration Plan 为配置计划; Visited node 为已访问节点, Pruned node 为已剪枝节点。

遵循上述三项原则, 我们提出概率工作流调度框架 Dyna。其主要组件如图 2 所示。框架由静态优化与动态优化组成。

当一个附有预定概率截止期保证的工作流提交给 WaaS 提供商时, 首先通过静态优化, 为每个任务确定费用效率最高的配置计划。为一个工作流生成配置计划的静态优化主要有两步: 先使用基于 A^* 的实例配置方法选择按需实例类型, 再通过混合实例配置细化考虑竞价实例。注意, 对同一工作流类, 即结构与任务剖析信息相同的工作流, 只需执行一次静态优化, 之后便可直接从配置计划缓存中读取计划。配置计划缓存用于保存已执行工作流类的配置计划。在运行时, 工作流任务根据各自实例配置调度, 并采用整合、实例复用等动态优化技术进一步降低成本。

本节其余部分介绍静态优化与动态优化的设计, 并讨论实现细节。

3.2 静态优化

静态优化的总体作用, 是为工作流中的每个任务确定合适的实例配置, 使费用尽量降低, 同时满足概率性能保证。理想情况下, 可以同时考虑按需实例与竞价实例, 但这样会使解空间过大。因此, 我们对静态优化采用分治思路, 将其分为基于 A^* 的实例配置和混合实例配置细化两类优化。基本思路是先为每个任务确定按需实例类型, 再引入混合执行配置来细化工作流执行方案, 进一步降低费用。具体而言, 将确定按需实例类型的过程形式化为一种基于 A^* 的方法; 再以 A^* 配置阶段的输出为基础, 为每个任务确定混合实例配置。

3.2.1 基于 A^* 的实例配置

在这一优化中, 我们为工作流的每个任务确定一种按需实例类型, 并将该过程形式化为一个基于 A^* 的搜索问题。选用 A^* 搜索, 是希望利用其剪枝能力缩小庞大的搜索空间, 同时获得高质量方案。

基于 A^* 的实例配置由经典 A^* 搜索过程扩展而来。在形式化的 A^* 搜索中, 状态定义为工作流的一个配置计划。为便于描述, 将配置计划表示为多维向量, 每一维对应工作流中一个任务的实例配置。向量的第 i 维, 对应 ID 为 i 的任务所分配的实例配置。这里有两点需要说明: 第一, 确保每个任务具有唯一 ID, ID 的分配顺序不影响算法正确性; 本文简单

采用拓扑序分配任务 ID。第二, 在这一静态优化阶段, 实例配置仅包含一个按需实例, 下一小节再将其扩展为混合实例配置。例如, 图 3 中的一个状态表示为 (t_0, t_1, t_2) , 其含义是任务 i ($0 \leq i \leq 2$) 被配置为使用 t_i 类型的按需实例。

最初的 A^* 搜索算法是一种启发式搜索方法, 用于寻找给定初始状态到指定目标状态的最短路径。搜索过程中, A^* 算法尝试计算当前已知最小成本, 并在每个状态剪去不必要的状态。具体而言, A^* 用两个距离指标 $g(s)$ 与 $h(s)$ 共同评估状态 s : 前者是初始状态到状态 s 的实际距离, 后者是状态 s 到目标状态的估计距离。 $g(s)$ 与 $h(s)$ 也分别称为状态 s 的 g 分数与 h 分数。状态 s 的总搜索成本估计为 $f(s) = g(s) + h(s)$ 。 A^* 搜索算法等价于一棵搜索树上进行搜索与剪枝。图 3 给出了本文问题设置下的配置计划搜索树示例。每个节点表示 A^* 搜索算法中的一个状态, 在本文中即工作流的一个配置计划。子状态与父状态仅有一维不同: 这一维被替换为更昂贵的实例类型。

算法 1 从初始状态 S 到目标状态 D 的、基于 A^* 的实例配置搜索

输入: Max_iter : 最大迭代次数;
 $deadline, pr$: 要求的概率截止期保证

```

1: ClosedList := 空;
2: OpenList :=  $S$ ;
3: upperBound := 0;
4:  $g[S] := 0$ ;
5:  $f[S] := g[S] + estimate\_h\_score(S)$ ;
6: 当非 (OpenList 为空 或 达到  $Max\_iter$ ) 执行
7:    $current :=$  OpenList 中  $f$  值最小的状态;
8:    $percentile := estimate\_performance(current, pr)$ ;
9:   若  $percentile \leq deadline$  则
10:     $current\_cost := estimate\_cost(current)$ ;
11:    若  $current\_cost < upperBound$  则
12:       $upperBound = current\_cost$ ;
13:       $D = current$ ;
14:    从 OpenList 移除  $current$ ;
15:    将  $current$  加入 ClosedList;
16:    对  $current$  的相邻状态中的每个  $neighbor$  执行
17:       $g[neighbor] := cal\_g\_score(neighbor, S)$ ;
18:       $f[neighbor] := g[neighbor] + estimate\_h\_score(neighbor)$ ;
19:      若  $f[neighbor] \geq upperBound$  或  $neighbor$  在 ClosedList 中 则
20:        continue;
21:      若  $neighbor$  不在 OpenList 中 则
22:        将  $neighbor$  加入 OpenList;
23: 返回  $D$ ;
```

本文基于 A^* 的算法, 目标是在搜索树上寻找费用最低、并且满足概率截止期保证的最优状态。算法 1 给出了基于 A^* 的实例配置算法的优化过程。算法利用 ClosedList 与 OpenList 两个列表结构维护搜索树中的全部状态。OpenList 存放当前为寻找目标状态而考虑的状态; ClosedList 存放在 A^* 搜索过程中无需再次考虑的状态。函数 $estimate_h_score$ 和 $estimate_g_score$ 分别返回状态的 h 与 g 分数; $estimate_cost$ 和 $estimate_performance$ 分别返回状态的费用估计与可行性估计。这四个函数的核心操作, 是估计一个状态, 即一个配置计划的执行时间和费用概率分布。

我们构建概率分布模型, 描述 I/O 与网络的性能动态性。已有研究 [13], [15] 表明, 由于资源共享, I/O 与网络是云中性能变化的主要来源, 而给定实例类型的 CPU 性能相对稳定。对于类型为 $type$ 的实例, 将其 I/O 或网络带宽等于某个数值 x 的概率定义为: $P_{seqBand, type}(seqBand = x)$ 、 $P_{rndBand, type}(rndBand = x)$ 、 $P_{inBand, type}(inBand = x)$ 和 $P_{outBand, type}(outBand = x)$ 。这四种概率分布, 分别表示 $type$ 类型实例的顺序 I/O、随机 I/O, 以及从持久存储下载和向

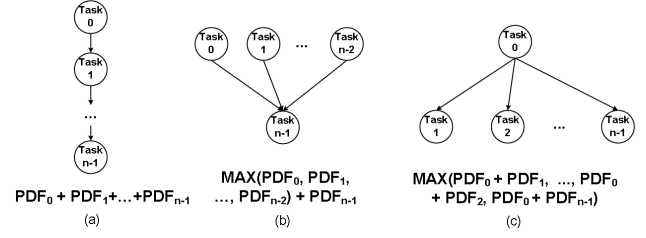


图 4. 基本工作流程结构及其执行时间概率分布; 任务 0、1、...、 $n-1$ 的执行时间分布分别记为 PDF_0 、 PDF_1 、...、 PDF_{n-1} 。图内 Task 为任务; PDF 表示概率分布函数, MAX 表示最大值分布运算, 各下标及公式沿用原图。

持久存储上传的网络性能。本文对 Amazon EC2 的校准显示, $P_{rndBand, type}(rndBand = x)$ 符合正态分布, 其余三种符合 Gamma 分布, 见第 4 节。给定 I/O 与网络性能分布, 以及 I/O 数据量和网络数据量, 便可用概率分布函数 (PDF) 描述任务在不同实例类型上的执行时间。

将任务执行时间建模为概率分布之后, 下面介绍任务执行成本的估计。考虑在价格为 p 、类型为 $type$ 的按需实例上执行的任务。任务期望费用估计为 p 乘以其在 $type$ 类型按需实例上的期望执行时间。整个工作流的期望费用是所有任务成本之和。这里的估计忽略了计费向上取整带来的费用, 因为在 WaaS 环境中, 这部分费用通常会被许多任务摊销。若严格施加实例小时计费模式, 会严重限制优化空间, 导致次优解。

另一项核心操作, 是判断配置计划是否满足概率截止期要求。给定待评估配置计划下每个任务的执行时间分布, 首先计算整个工作流的执行时间分布。具体做法是将工作流分解为图 4 所示的三种基本结构, 每种结构包含 n 个任务 ($n \geq 2$)。各基本结构的执行时间分布, 由其中各个任务的执行时间分布计算得到。例如, 图 4(b) 中结构的执行时间分布计算为 $MAX(PDF_0, PDF_1, \dots, PDF_{n-2}) + PDF_{n-1}$, 其中 PDF_i ($0 \leq i \leq n-1$) 是任务 i 的执行时间概率分布。两个概率分布之间的“+”运算, 计算这两个分布的卷积; MAX 运算, 则求出这两个分布所描述随机变量的最大值分布。得到工作流执行时间分布后, 检查要求的概率截止期保证所对应的百分位。只有返回的百分位小于截止期, 所评估的配置计划才是可行的。

将 A^* 搜索的初始状态设为配置 $(0, 0, \dots, 0)$, 其中每个任务都使用最便宜的实例类型, 即实例类型 0。目标状态是最优配置计划。 $g(s)$ 定义为状态 s 与初始状态之间的费用差。

将 $h(s)$ 估计为配置计划 s 的费用。搜索过程中, 将已经找到的最低成本维护为上界, 用于剪去搜索树中无用的状态。注意, 只考虑满足概率截止期保证的可行状态。树的叶节点包含一个工作流的所有可能配置计划。搜索从初始配置开始, 通过为状态扩展子状态构建搜索树。在第 l 层, 扩展操作将状态的第 l 维替换为所有可能的、更昂贵的实例类型。因此, 所有子状态的执行时间分布都优于其祖先状态。如果状态 s 的费用高于上界, 则其后继不太可能成为目标状态, 因为后继配置比 s 更昂贵。例如, 假设图 3 搜索树中的配置 $(1, 1, 0)$ 费用高于上界, 图中灰色节点便可在 A^* 搜索中剪去。

3.2.2 混合实例配置细化

我们将使用竞价实例视为对基于 A^* 的实例配置算法所生成计划的细化, 以进一步降低费用。使用竞价实例的主要问题, 是任务可能遭遇出价失效事件, 从而无法满足截止期要求。为处理这一问题, 本文提出同时使用按需实例与竞价实例的混合实例配置。基本思路是: 若截止期允许, 在任务使用按需实例之前, 先尝试在竞价实例上运行。若任务能在竞价实例上完成, 费用往往更低。如果先前的竞价实例失败, 只要仍可降低费用并满足概率性能保证, 就可以继续尝试一个以上的竞价实例。若混合配置中的所有竞价实例均失败, 则在按需实

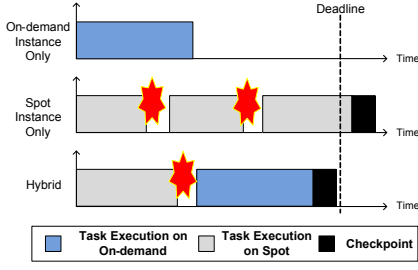


图 5. 混合方法示意图内三行依次为仅使用按需实例、仅使用竞价实例和混合方式；Time 为时间，Deadline 为截止期；图例依次为在按需实例上执行任务、在竞价实例上执行任务，以及检查点。

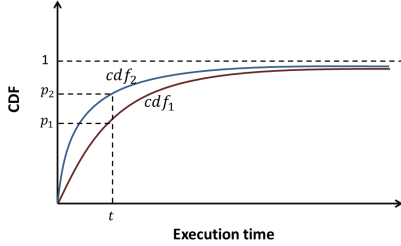


图 6. 配置 $C_2 \geq C_1$ 的定义。 cdf_1 与 cdf_2 分别是配置计划 C_1 与 C_2 下执行时间的累积分布函数。图内 Execution time 为执行时间，CDF 为累积分布函数。

例上执行任务，以确保截止期。图 5 比较了单个任务的按需、竞价与混合配置。任务在竞价实例上失败，不会触发其前驱任务重新执行：前驱任务的结果已做检查点保存，并物化到云中的持久存储，例如 Amazon S3。Dyna 仅在任务结束时做检查点，这一方法简单，其开销远低于通用检查点算法 [19]。

如第 2.2 节所述，混合实例配置表示为同时包含竞价与按需实例类型的向量。向量最后一维，是基于 A^* 的实例配置阶段得到的按需实例类型。初始混合配置只包含一个按需实例类型，之后反复在配置前面加入竞价实例，寻找最优混合配置。将任务 T 的混合配置 C_1 细化为 C_2 时，需要判断 C_2 的执行时间分布是否优于 C_1 。具体而言，若对 $\forall t$ 都有 $\int_0^t P_{T,C_2}(time = x) dx \geq \int_0^t P_{T,C_1}(time = x) dx$ ，则定义 $C_2 \geq C_1$ ；其中， P_{T,C_1} 与 P_{T,C_2} 分别是任务 T 在混合配置 C_1 和 C_2 下的 PDF。图 6 展示了这一定义，积分以 CDF，即累积分布函数表示。

理想情况下，可以加入 n 个竞价实例，其中 n 是预定义参数。较大的 n 会提高从竞价实例中获益的概率；较小的 n 则会提高满足截止期要求的概率，并降低优化开销。实验中发现， $n = 2$ 已足以获得良好的优化结果。更大的 n 会显著增加优化开销，却只能小幅改善优化结果。

对混合实例配置中新加入的每个竞价实例，需要决定其类型与相应出价。由于竞价价格具有动态性，这一决策并不简单。我们按照算法 2 搜索竞价实例类型及相应出价。为缩小搜索空间，设计如下启发式：为了确保概率截止期保证，新加入的竞价实例类型应至少与混合配置中的按需实例类型同样昂贵。对每个竞价实例类型，使用算法 3 在 $[P_{min}, P_{max}]$ 范围内搜索出价。实现中， P_{min} 为 0.001， P_{max} 等于同类型按需实例的价格。出价高于 P_{max} 的竞价实例，无助于降低费用。若为某一实例类型找到合适出价，便将该类型及找到的出价写入混合实例配置。若所有竞价实例类型都没有合适出价，则将新加入的竞价实例类型设为 -1，表示未加入竞价实例。

算法 3 使用二分搜索算法寻找出价，考虑两项要求：第一，加入竞价实例不得违反概率截止期保证；第二，细化后的混合配置估计成本，应低于细化前的估计成本。若某一出价同时满足这两项要求，就将该价格返回给混合实例配置。

概率截止期保证的考虑。为了确保概率截止期保证，一种

算法 2 任务 T 的混合实例配置细化。

输入： $ondemand_type$ ：为任务 T 在 A^* 实例配置阶段找到的按需类型
 n ：混合实例配置的维数
1: $T.configList[n] := ondemand_type$;
2: $T.prices[n] := ondemand_type$ 的按需价格;
3: 对 $dim := 1$ 至 $n - 1$ 执行
4: $T.configList[dim] := -1$;
5: $T.prices[dim] := 0$;
6: 对 $dim := 1$ 至 $n - 1$ 执行
7: 对 $spottype := ondemand_type$ 至 最昂贵的实例类型 执行
8: $P_{max} :=$ 实例类型 $spottype$ 的按需价格;
9: $P_b := binary_search(P_{min}, P_{max}, T, spottype)$;
10: 若 $P_b == -1$ 则
11: continue;
12: 否则
13: $T.configList[dim] := spottype$;
14: $T.prices[dim] := P_b$;

算法 3 任务 T 的 Binary_search 函数。

输入： P_{low} ：搜索的最低出价
 P_{high} ：搜索的最高出价
 $spottype$ ：竞价实例类型
1: 若 $P_{low} > P_{high}$ 则
2: 返回 -1;
3: $P_{mid} := (P_{low} + P_{high})/2$;
4: $spotcost := estimate_cost$ (出价为 P_{mid} 的混合配置);
5: $ondemandcost := estimate_cost$ (按需配置);
6: 若 $spotcost > ondemandcost$ 则
7: 返回 $binary_search(P_{low}, P_{mid}, spottype)$;
8: 否则
9: $satisfied := estimate_performance$ (混合配置, T);
10: 若非 $satisfied$ 则
11: 返回 $binary_search(P_{mid}, P_{high}, spottype)$;
12: 返回 P_{mid} ;

做法是计算整个工作流执行时间的概率分布，再判断要求的百分位是否小于截止期。然而，这种计算的开销很大。第 4 节介绍的 Oracle 算法实现了这一过程，而 Dyna 使用轻量的局部启发式以降低开销。由于基于 A^* 的实例配置阶段所找到的按需配置，已经确保概率截止期要求，因此只需保证每个任务的混合配置 C_{hybrid} 满足 $C_{hybrid} \geq C_{ondemand}$ ；其中， $C_{ondemand}$ 是仅含按需实例的配置。若不满足这一要求，则意味着当前出价太低，继续在当前搜索范围的较高半区搜索出价。

竞价实例可能在任意时刻失败，因此定义概率函数 $ffp(t, p)$ ，计算当出价设为 p 时，竞价实例首次在时刻 t 失败的概率。已有研究表明，可利用统计模型 [27] 或逆向分析 [26] 预测竞价价格。本文以近期的竞价价格历史预测实际竞价价格，并供 $ffp(t, p)$ 计算失败概率。我们使用基于 Monte Carlo 的方法得到该函数：从价格历史中的随机位置开始，若价格首次在时刻 t 高于 p ，则将计数器 $count$ 加一。重复该过程 NUM 次，其中 NUM 足够大，再返回 $\frac{count}{NUM}$ 作为失败概率。

为了判断细化后的混合实例配置是否更好，首先讨论如何在给定混合配置下估计任务的总体执行时间分布。假设任务 T 的混合配置为 $C_{hybrid} = \langle (type_1, P_b, isSpot), (type_2, P_o, isOndemand) \rangle$ 。任务 T 在 $type_1$ 类型竞价实例上的概率分布为 $P_{T,type_1}$ ，在 $type_2$ 类型按需实例上的概率分布为 $P_{T,type_2}$ 。任务 T 在 C_{hybrid} 下的总体执行时间可分为两种情形。若任务在竞价实例上成功完成，则总体执行时间等于任务 T 在该竞价实例上的执行时间 t_s ，对应概率如下。

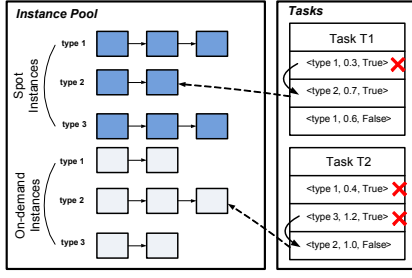


图 7. 混合执行示例图内左侧为实例池，包括竞价实例与按需实例，各 type 1、type 2、type 3 为类型 1、2、3；右侧为任务 T1、T2。配置中的 True 表示竞价实例，False 表示按需实例，数字表示类型或出价。

$$P_{T, C_{hybrid}}(time = t_s) = P_{T, type_1}(time = t_s) \times (1 - \int_0^{t_s} ffp(x, P_b) dx) \quad (1)$$

否则，总体执行时间等于任务 T 在竞价实例上运行至失败所经历的时间 t_f ，加上任务 T 在按需实例上的执行时间 t_o ，对应概率如下。

$$P_{T, C_{hybrid}}(time = t_f + t_o) = P_{T, type_1}(time = t_f) \times P_{T, type_2}(time = t_o) \times ffp(t_f, P_b)(t_f \leq t_s) \quad (2)$$

得到任务在混合配置 C_{hybrid} 下的执行时间分布后，使用图 6 中的定义，将其与按需配置 $C_{ondemand}$ 比较。若满足 $C_{hybrid} \geq C_{ondemand}$ ，则接受任务细化后的混合实例配置。通过这一启发式，将整个工作流执行时间分布的计算局部化为逐任务计算，显著降低优化开销。

费用的考虑。若细化配置的期望费用高于细化前，意味着出价设得太高。因此，继续在当前搜索范围的较低半区搜索出价。算法 4 说明了任务混合配置成本的估计。与性能估计的分析类似，我们得到混合配置的执行时间分布，并计算所搜索出价下的期望费用。比较细化前后混合配置的期望费用，只有细化能够降低期望费用时，才加入竞价实例。

算法 4 估计具有 2 个实例的混合配置任务的费用 C

输入： 任务的混合实例配置： $\langle type_1, P_b, isSpot \rangle$ 与 $\langle type_2, P_o, isOndemand \rangle$;
 任务在 $type_1$ 实例上的执行时间样本： $stime_i$ ($1 \leq i \leq N$);
 任务在 $type_2$ 实例上的执行时间样本： $otime_i$ ($1 \leq i \leq N$);

- 1: $C = 0$;
- 2: **对** $i = 1$; $i \leq N$; $i = i + 1$ **执行**
- 3: $p = 0$;
- 4: **对** $t = 0$; $t < stime_i$; $t = t + step$ **执行**
- 5: $p = p + ffp(t, P_b)$;
- 6: $C = C + P_b \times stime_i + p \times P_o \times otime_i$;
- 7: $C = \frac{C}{N}$;
- 8: 返回 C ;

3.3 动态优化

当工作流中某个任务的所有前驱任务均执行完成时，该任务进入就绪状态，可以按照静态优化所确定的实例配置，调度到实例上执行。在动态优化阶段，我们采用整合与伸缩技术，并实现在线实例复用，以进一步优化执行成本。这些技术已被以往研究广泛采用 [3], [12]，但仍需扩展，使之考虑混合实例配置。下面简要介绍这些技术，并特别说明支持混合实例配置带来的差异。

整合与伸缩。整合不同实例可以减少新实例的获取时间，以及按实例小时计费的额外开销。对实例池中的每个实例，记录其当前不足一小时部分的剩余时间。

由于竞价实例与按需实例不同，对二者进行虚拟机整合时需要特别处理。若一个竞价实例或按需实例的剩余时间足以执行某个就绪任务，而该任务本来需要获取同类型竞价实例或按需实例，就将任务分配给该实例。由于成功执行保证的要求，竞价实例与按需实例之间的整合只允许一种情况：将原本请求竞价实例的任务整合到按需实例上。

在线实例复用。运行时维护正在运行的实例池，按不同实例类型组织成列表。同类型的竞价实例与按需实例，分别存入不同列表。在运行时，处理一个实例请求时，先在实例启动时刻检查实例池中是否存在所需类型的空闲实例。若存在，则选择它执行工作流；否则，从 IaaS 云获取该类型的新实例。这样，在工作流执行过程中启动的实例便能得到适当复用，提高利用率。此外，如果能够复用实例，也就消除了实例获取时间。

图 7 给出了当前竞价实例和按需实例池的示例，以及两个待执行任务 T1 和 T2。最初，任务 T1 分配到 $type_1$ 类型竞价实例，但任务 T1 因出价失效事件执行失败，随后在 $type_2$ 类型竞价实例上重新开始。若实例池中无可用的 $type_2$ 类型竞价实例，任务将等待新 $type_2$ 类型竞价实例建立，并按任务混合实例配置中的价格为该实例出价。同时，T2 已在 $type_1$ 与 $type_3$ 类型竞价实例上失败，因此将在 $type_2$ 类型按需实例上执行。类似地，若池中无可用的 $type_2$ 类型按需实例，则先请求一个新的 $type_2$ 类型按需实例，并将其加入实例池。

4 评估

本节介绍所提方法在 Amazon EC2 上的评估结果。

4.1 实验设置

我们在真实云环境中开展两组实验：首先，从 Amazon EC2 校准云动态性，作为优化框架的输入；其次，在 Amazon EC2 上使用待比较算法运行真实科学工作流，进行评估。

校准。对四种常用实例类型测量 CPU、I/O 和网络性能，即 m1.small、m1.medium、m1.large 和 m1.xlarge。我们发现，CPU 性能较稳定，与已有研究 [13] 一致。因此重点校准 I/O 和网络性能。具体而言，对每种实例重复性能测量 1,000 次。当实例已获取满一小时后，释放该实例，再创建同类型新实例继续测量。测量结果用于建立 I/O 与网络性能的概率分布模型。

对本地磁盘，同时测量顺序与随机 I/O 性能。使用 *hdparm* 测量顺序 I/O 读取性能；通过生成每次 512 字节的随机 I/O 读取，测量随机 I/O 性能。读与写的性能结果相近，因此本文不作区分。

我们测量不同实例类型与 Amazon S3 之间的上传及下载带宽，方法是向 S3 上传文件并从 S3 下载文件，文件大小设为 8MB。还使用 Iperf [47] 测量两个实例之间的网络带宽。结果显示，不同类型实例之间的网络带宽，通常低于同类型实例与 S3 之间的带宽。

工作负载。已有研究分析科学工作流的性能行为 [48], [49]。本文考虑三种常见工作流结构：Ligo、Montage 和 Epigenomics。Ligo，即激光干涉引力波天文台，是用于探测引力波的应用；Montage 是广泛用作网格与并行计算基准的天文学应用；Epigenomics 是包含多种基因组测序操作的数据处理流水线。如图 8 所示，三种工作流具有不同结构与并行度，对计算资源的要求也不同。例如，Montage 是 I/O 密集型负载，Ligo 是内存密集型负载，Epigenomics 是 CPU 密集型负载。因此，使用这三种工作流，可以检验所提算法对不

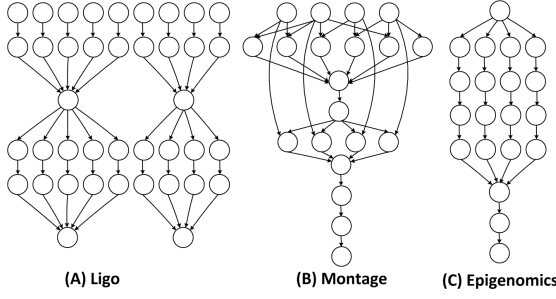


图 8. workflow 结构：Ligo、Montage 和 Epigenomics。

同负载的有效性。Juve 等 [48] 已对这些 workflow 进行详细分析，更多信息可参阅其研究。

比较。为评估 Dyna 中所提技术的有效性，我们在 Amazon EC2 上实现以下算法。

- **Static.** 该方法与已有研究 [3] 相同，仅采用按需实例，作为先进方法的比较基线。为公平比较，依据 Dyna 所使用的 QoS 设置来确定 workflow 截止期。例如，若用户要求 90% 的概率截止期保证，则将实验中的确定性截止期设为其执行时间分布的第 90 百分位。
- **Dyna-NS.** 除完全不使用竞价实例外，该方法与 Dyna 相同。比较 Dyna 与 Dyna-NS，是为了评估竞价实例的影响。
- **SpotOnly.** 该方法在执行期间仅采用竞价实例。它先利用基于 A^* 的实例配置方法，为 workflow 中每个任务决定实例类型；随后将每个任务的出价设得很高，在本文研究中设为 \$1,000，以保证要求的截止期满足率。
- **Oracle.** 我们实现 Oracle 方法，以评估 Dyna 优化开销与优化有效性之间的权衡。Oracle 与 Dyna 的关键区别如下：在为任务混合配置中的竞价实例搜索出价的二分搜索方法里，Oracle 不使用 Dyna 的局部百分位计算启发式。相反，对于某个任务的每个候选出价，它都计算整个 workflow 的执行时间分布，判断该出价是否满足概率截止期要求。由于 Oracle 获得方案所需的时间开销过高，它是一种离线方法。

我们采用 DAGMan [50] 管理任务依赖，并在 Amazon 机器映像 (AMI) 中加入 Condor [51]，管理任务执行与实例获取。

实例从美国东部区域获取。workflow 执行所用四种实例类型的按需小时费用，见表 1。这四种实例也被已有研究 [19] 采用。对于实例获取时间 lag ，实验显示，获取每个按需实例平均需要 2 分钟，获取竞价实例平均需要 7 分钟，与 Mao 等 [52] 的已有研究一致。

workflow 截止期是决定实例配置候选空间的重要因素。这里特别关注两种截止期设置： D_{min} 和 D_{max} ，分别表示 workflow 关键路径上的所有任务均在 m1.xlarge 与 m1.small 实例上执行时的期望执行时间。默认截止期设为 $\frac{D_{min} + D_{max}}{2}$ 。

假设用户向 WaaS 提供商提交许多 workflow。每次实验向云提交 100 个具有相同 workflow 结构的作业，假设作业到达服从 Poisson 分布。Poisson 分布的参数 λ 会影响虚拟机复用的机会，默认将 λ 设为 0.1。

评估指标为单个 workflow 的平均费用与经历时间。所有指标均以 Static 对应指标为基准归一化。给定概率截止期要求，多次在云上运行待比较算法，记录其费用和执行时间。当各算法均满足 QoS 要求时，费用是比较不同调度算法优化有效性的主要指标。默认要求的截止期满足率设为 96%，所报告结果也默认使用全部参数的默认设置。第 4.4 节通过敏感性实验研究各参数的影响。

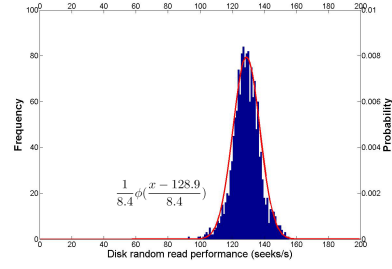


图 9. Medium 实例随机 I/O 性能的直方图与概率分布图内横轴为磁盘随机读取性能（每秒寻道次数），左纵轴为频数，右纵轴为概率。

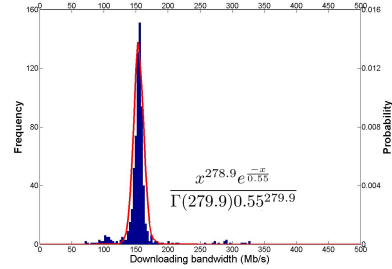


图 10. Medium 实例与 S3 存储之间下载带宽的直方图与概率分布图内横轴为下载带宽（Mb/s），左纵轴为频数，右纵轴为概率。

4.2 云动态性

本小节报告在 Amazon EC2 中观察到的性能动态性。价格动态性已在表 1（第 2 节）中给出。

图 9 和图 10 分别给出 Medium 实例的随机 I/O 性能，以及从 Amazon S3 下载的网络性能测量结果。其他实例类型也观察到类似结果，据此得到以下观察。

首先，I/O 与网络性能都可以用正态分布或 Gamma 分布建模。我们通过零假设检验验证这些分布，发现：（1）四种实例类型的顺序 I/O 性能，以及与 S3 之间的上传、下载网络带宽，均服从 Gamma 分布；（2）四种实例类型的随机 I/O 性能分布，均服从正态分布。分布参数列于表 2 和表 3。

其次，同一实例类型的 I/O 与网络性能变化显著，m1.small 和 m1.medium 尤为明显。这可以从表 2 和表 3 中 Gamma 分布的 θ 参数、或正态分布的 σ 参数看出。此外，同类型实例的随机 I/O 性能变化，比顺序 I/O 性能更明显。m1.small 的顺序 I/O 和随机 I/O 性能变异系数，分别为 9% 与 33%。

第三，不同实例类型之间的性能也有很大差异。这可以从表 2 和表 3 中 Gamma 分布的 $k \cdot \theta$ ，即期望值，或正态分布的 μ 参数看出。

表 2
I/O 性能分布参数

实例类型	顺序 I/O (Gamma)	随机 I/O (正态)
m1.small	$k = 129.3, \theta = 0.79$	$\mu = 150.3, \sigma = 50.0$
m1.medium	$k = 127.1, \theta = 0.80$	$\mu = 128.9, \sigma = 8.4$
m1.large	$k = 376.6, \theta = 0.28$	$\mu = 172.9, \sigma = 34.8$
m1.xlarge	$k = 408.1, \theta = 0.26$	$\mu = 1034.0, \sigma = 146.4$

表 3
实例与 S3 之间带宽的 Gamma 分布参数

实例类型	上传带宽	下载带宽
m1.small	$k = 107.3, \theta = 0.55$	$k = 51.8, \theta = 1.8$
m1.medium	$k = 421.1, \theta = 0.27$	$k = 279.9, \theta = 0.55$
m1.large	$k = 571.4, \theta = 0.22$	$k = 6187.7, \theta = 0.44$
m1.xlarge	$k = 420.3, \theta = 0.29$	$k = 15313.4, \theta = 0.23$

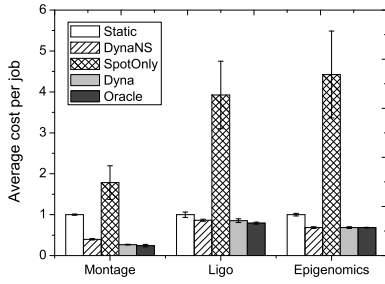


图 11. Amazon EC2 上各算法处理 Montage、Ligo 和 Epigenomics 工作流时的平均费用优化结果。图内纵轴为每作业平均费用，横轴为三种工作流；图例 Static、DynaNS、SpotOnly、Dyna、Oracle 为本文所定义的算法名称。

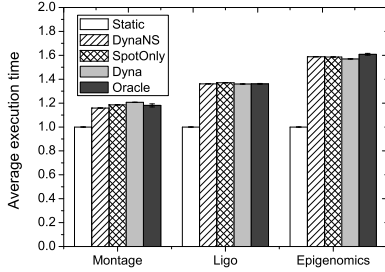


图 12. Amazon EC2 上各算法处理 Montage、Ligo 和 Epigenomics 工作流时的平均执行时间优化结果。图内纵轴为平均执行时间，横轴为三种工作流；图例沿用算法名称。

4.3 总体比较

本节给出默认设置下，Dyna 与其他算法在 Amazon EC2 上的总体比较结果；敏感性研究见第 4.4 节。表 4 列出实际获得的截止期满足率。注意，我们使用第 4.2 节的校准结果，作为 Dyna 优化的输入。结果表明，所设计技术能够准确满足要求的概率截止期保证。虽然 Static 也能以更高的截止期满足率达到要求，但会给 WaaS 提供商带来高得多的额外费用，本小节随后将展示这一点。

图 11 给出 Static、DynaNS、SpotOnly、Dyna 和 Oracle 方法，在 Montage、Ligo 和 Epigenomics 负载上的每作业平均

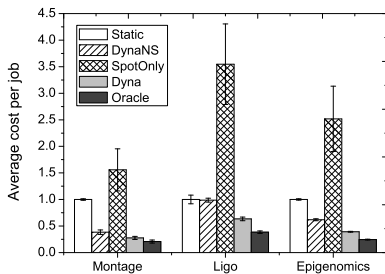


图 13. 概率截止期保证为 90% 时，各算法处理 Montage、Ligo 和 Epigenomics 工作流的平均费用结果。图内纵轴为每作业平均费用，横轴为三种工作流；图例沿用算法名称。

表 4

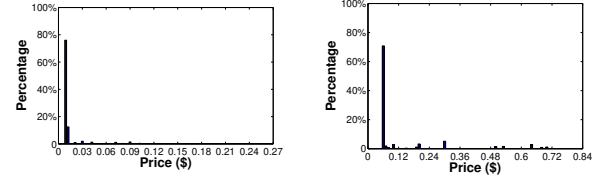
概率截止期保证为 96% 时，各算法在 Montage、Ligo 和 Epigenomics 工作流上获得的截止期满足率。

	Static	DynaNS	SpotOnly	Dyna	Oracle
Montage	100%	96.5%	96.7%	96.7%	96.6%
Ligo	100%	96.2%	96.8%	96.8%	96.6%
Epigenomics	100%	96.8%	96.7%	96.8%	96.7%

表 5

各算法在 Montage、Ligo 和 Epigenomics 工作流上的优化开销，单位为秒。

	Static	DynaNS	SpotOnly	Dyna	Oracle
Montage	1	153	153	163	2997
Ligo	1	236s	236	244	10452
Epigenomics	1	166	166	175	2722



(a) m1.small 实例类型竞价价格的直方图 (b) m1.xlarge 实例类型竞价价格的直方图

图 14. Amazon EC2 美国东部区域 2013 年 8 月历史竞价价格的直方图。两幅子图的横轴均为价格（美元），纵轴均为百分比。

费用。在所测试负载中，这五种方法费用结果的标准误差，依次为 0.01–0.06、0.02–0.04、0.40–0.76、0.01–0.03 和 0.01–0.03。总体而言，在三个负载中，Dyna 都是在线方法中费用最低的；相比 Static、DynaNS 和 SpotOnly，分别节省 15–73%、1–33% 和 78%–85% 的费用。我们得到以下观察。

首先，DynaNS 的费用低于 Static，因为所提 A* 配置搜索技术能够找到更便宜的实例配置，并适用于不同工作流结构。这也说明，Static 在实例配置之前先分配任务截止期，会降低优化有效性。

其次，Dyna 的费用低于 DynaNS，表明采用竞价实例的混合配置能够有效降低费用。概率截止期保证设得越低，Dyna 相对 DynaNS 节省的费用越多。图 13 给出概率截止期保证设为 90% 时各算法的费用。在该设置下，Dyna 比 DynaNS 节省 28–37% 的费用。

第三，SpotOnly 在所有待比较算法中费用最高，原因是竞价价格的动态特性。图 14 给出了实验月份的竞价价格直方图。虽然竞价价格大部分时间低于同类型按需价格，但某些时刻可能远高于按需价格。如表 1 所示，2013 年 8 月 m1.small 实例的最高竞价价格为 \$10，是其按需价格的 160 多倍。SpotOnly 相对其他算法的费用，在 Ligo 上尤其高，因为 Ligo 的平均执行时间长于其他工作流。不过，这一观察依赖竞价价格的波动。如果在其他时段运行实验，SpotOnly 与 Dyna 的比较结果可能不同。第 4.4 节将使用另一段竞价价格历史，研究二者对竞价价格的敏感性。

图 12 给出了 Static、DynaNS、SpotOnly、Dyna 和 Oracle 方法，在 Amazon EC2 上处理 Montage、Ligo 和 Epigenomics 负载时单个工作流的平均执行时间。各算法执行时间结果的标准误差都较小，在所测试负载上均为 0.004–0.01。Static 的平均执行时间最短，因为它为工作流中每个任务配置性能更好、价格更高的实例类型。SpotOnly、Dyna 和 Oracle 的平均执行时间相近，因为三者都使用本文的静态优化，为工作流中每个任务配置某种实例类型。此外，Dyna 对各任务出价的仔细选择，以及 SpotOnly 的高出价，减少了执行期间的出价失效事件。另一方面，也可看出，静态优化期间搜索得到的出价，能够减少出价失效事件。

最后，分析各算法的优化开销，结果见表 5。注意，对结构与剖析信息相同的工作流，本文框架仅需优化一次。虽然 Oracle 的费用低于 Dyna，但其优化开销是 Dyna 的 16–44 倍。这说明 Dyna 能在短得多的时间内，找到接近最优的优化结果。由于 Oracle 的优化开销较大，后续实验不再评估 Oracle，仅比较 Dyna、Static、DynaNS 和 SpotOnly。

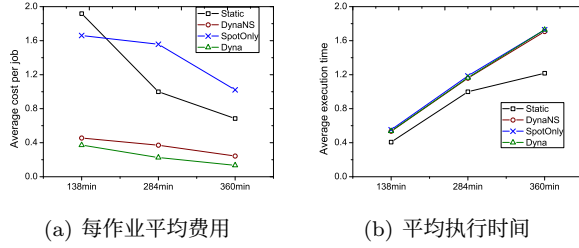


图 15. 截止期敏感性研究中的平均费用与平均执行时间结果。图内横轴刻度 138min、284min、360min 分别为 138、284、360 分钟；左图纵轴为每作业平均费用，右图纵轴为平均执行时间；图例沿用算法名称。

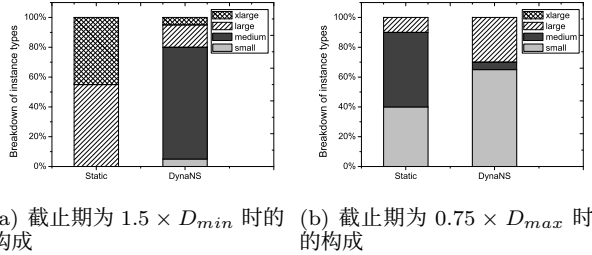


图 16. 截止期为 $1.5 \times D_{min}$ 与 $0.75 \times D_{max}$ 时，各算法采用实例类型的构成。图内纵轴为实例类型的构成比例，横轴为 Static 与 DynaNS；图例 xlarge、large、medium、small 分别为特大型、大型、中型、小型实例。

4.4 敏感性研究

我们对不同工作流开展敏感性研究。由于各工作流呈现类似结果，以下重点讨论 Montage 工作流。每项研究只改变一个参数，其余参数保持默认设置。

截止期。 截止期是决定实例配置的重要因素。我们将截止期要求依次设为 $1.5 \times D_{min}$ 、 $0.5 \times (D_{min} + D_{max})$ 与 $0.75 \times D_{max}$ ，评估各算法；对 Montage 工作流，它们平均分别对应 138、284 和 360 分钟。所有结果都以截止期为 $0.5 \times (D_{min} + D_{max})$ 时 Static 的结果归一化。图 15 给出每作业平均费用与平均执行时间。在所有测试截止期设置下，Dyna 的平均费用均为各算法中最低。截止期越宽松，费用越低，因为执行时可以使用更多便宜实例，即按需实例。这一趋势不适用于 SpotOnly，因为 m1.medium 的竞价价格有时可能低于 m1.small；研究价格轨迹后，我们验证了这一现象。进一步地，对截止期为 $1.5 \times D_{min}$ 与 $0.75 \times D_{max}$ 时，所使用不同类型按需实例和竞价实例的数量进行分解，见图 16。SpotOnly 和 Dyna 的类型构成与 DynaNS 相同，因为三者均采用基于 A^* 的实例配置方法。截止期宽松时，即 $0.75 \times D_{max}$ ，会使用更多便宜实例。此外，在相同截止期下，例如 $1.5 \times D_{min}$ ，DynaNS、SpotOnly 和 Dyna 比 Static 使用更多便宜实例，再次说明本文的 A^* 方法优于已有启发式 [3]。

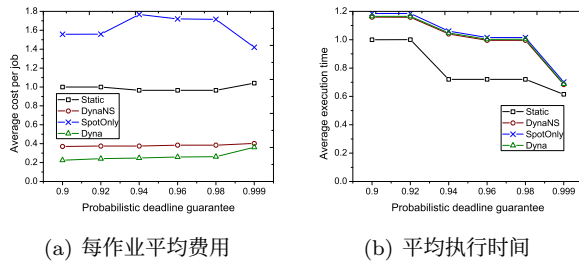


图 17. 概率截止期保证敏感性研究中的平均费用与平均执行时间结果。两幅子图横轴均为概率截止期保证；左图纵轴为每作业平均费用，右图纵轴为平均执行时间；图例沿用算法名称。

表 6
要求的截止期满足率变化时，各算法实际获得的截止期满足率。

要求的截止期满足率	实际截止期满足率			
	Static	DynaNS	SpotOnly	Dyna
90%	98.7%	90.7%	90.6%	90.7%
92%	98.7%	92.4%	92.3%	92.5%
94%	99.9%	94.4%	94.4%	94.3%
96%	99.9%	96.6%	96.6%	96.6%
98%	99.9%	98.2%	98.2%	98.0%
99.9%	100%	100%	100%	100%

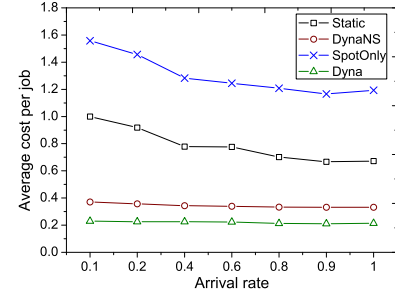


图 18. 工作流到达率敏感性研究中的平均费用与平均执行时间结果。图内横轴为到达率，纵轴为每作业平均费用；图例沿用算法名称。

概率截止期保证。 我们将概率截止期要求从 90% 变到 99.9%，评估 Dyna 满足这些要求的有效性。图 17 给出各算法的每作业平均费用与平均执行时间。对不同概率截止期保证设置，Dyna 的费用均最低；要求的保证概率越低，Dyna 节省的费用越多。表 6 给出要求的截止期满足率变化时，各算法实际获得的满足率。DynaNS、SpotOnly 和 Dyna 能够准确满足概率截止期保证，而 Static 不能。

到达率。 我们将工作流到达率 λ 依次设为 0.1、0.2、0.4、0.6、0.8、0.9 和 1.0，评估 Dyna 的有效性。所有结果以到达率为 0.1 时的结果归一化。图 18 给出优化后的每作业平均费用。在所有作业到达率下，Dyna 都获得最低平均费用。随着作业到达率提高，每作业平均成本下降，因为更多作业共享 WaaS 提供商从云中租用的资源，也共同分摊 IaaS 云按小时收取的实例费用。

竞价价格。 为了研究 Dyna 与 SpotOnly 对竞价价格变化的敏感性，使用仿真比较各算法在不同历史竞价价格下的表现。具体而言，采用亚太区域 2011 年 12 月的历史竞价价格。如表 7 所示，相比真实实验所用的 2013 年 8 月，该时段价格很低且稳定。因此，竞价实例在执行期间较少失败，失败概率 ffp 很低。我们推测，2013 年 8 月的价格波动比 2011 年 12 月更大，是因为使用 Amazon EC2 竞价实例的用户更多。图 19 给出实际得到的费用结果。SpotOnly 与 Dyna 的费用相

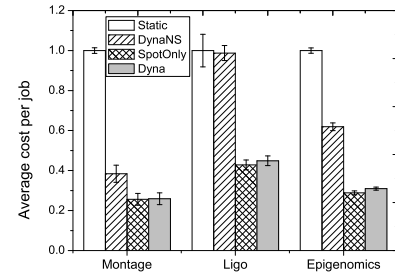


图 19. 使用 Amazon EC2 亚太区域 2011 年 12 月历史竞价价格时，各算法平均费用的仿真结果。图内横轴为三种工作流，纵轴为每作业平均费用；图例沿用算法名称。

表 7
Amazon EC2 竞价价格统计 (美元/小时, 2011 年 12 月, 亚太区域) 及
按需价格。

实例类型	均值	stdev	最小值	最大值	按需价格
m1.small	0.041	0.003	0.038	0.05	0.06
m1.medium	0.0676	0.003	0.064	0.08	0.12
m1.large	0.160	0.005	0.152	0.172	0.24
m1.xlarge	0.320	0.009	0.304	0.336	0.48

近, 且都远低于 Static 和 DynaNS。这表明, 对不同竞价价格分布, Dyna 都能够在费用优化上优于 SpotOnly。

5 结论

随着云中各种科学应用与数据密集型应用日益普及, 在 IaaS 云中托管 WaaS 正逐渐兴起。然而, IaaS 云是性能与价格都会变化的动态环境, 使静态任务执行时间假设以及以确定性截止期定义 QoS 的方式都不再合适。本文提出在动态云环境中, 将概率性能保证作为 QoS。我们构建名为 Dyna 的科学工作流概率调度框架, 目标是在满足概率截止期保证的同时最小化费用。框架包含一系列静态与动态优化, 并进一步针对价格动态性, 构建竞价实例与按需实例的混合实例配置。我们在 Amazon EC2 上部署 Dyna, 并使用真实科学工作流评估其有效性。实验表明, Dyna 能在准确满足用户概率要求的同时, 将费用相对于先进方法显著降低, 降幅达 73%。

参考文献

- [1] Amazon Case Studies, “<http://aws.amazon.com/solutions/case-studies/>”
- [2] Windows Azure Case Studies, “<http://www.microsoft.com/azure/casestudies.mspx>”
- [3] M. Mao and M. Humphrey, “Auto-scaling to minimize cost and meet application deadlines in cloud workflows,” in *SC '11*, 2011, pp. 49:1–49:12.
- [4] H. Kilapi, E. Sitaridi, M. M. Tsangaris, and Y. Ioannidis, “Schedule optimization for data processing flows on the cloud,” in *SIGMOD '11*, 2011, pp. 289–300.
- [5] J. Yu, R. Buyya, and C. K. Tham, “Cost-based scheduling of scientific workflow application on utility grids,” in *e-Science '05*, 2005, pp. 140–147.
- [6] R. Sakellariou, H. Zhao, E. Tsiakkouri, and M. D. Dikaiakos, “Scheduling workflows with budget constraints,” in *CoreGRID '05*, 2005.
- [7] R. Duan, R. Prodan, and T. Fahringer, “Performance and cost optimization for multiple large-scale grid workflow applications,” in *SC '07*, 2007, pp. 12:1–12:12.
- [8] S. Abrishami, M. Naghibzadeh, and D. Epema, “Deadline-constrained workflow scheduling algorithms for iaas clouds,” in *Future Gener. Comput. Syst.*, 2013, pp. 158–169.
- [9] E.-K. Byun, Y.-S. Kee, J.-S. Kim, and S. Maeng, “Cost optimized provisioning of elastic resources for application workflows,” in *Future Gener. Comput. Syst.*, 2011, pp. 1011–1026.
- [10] S. T. Maguluri, R. Srikant, and L. Ying, “Stochastic models of load balancing and scheduling in cloud computing clusters,” in *INFOCOM '12*, 2012.
- [11] F. Zhang, J. Cao, K. Hwang, and C. Wu, “Ordinal optimized scheduling of scientific workflows in elastic compute clouds,” in *CLOUDCOM '11*, 2011.
- [12] M. Malawski, G. Juve, E. Deelman, and J. Nabrzyski, “Cost- and deadline-constrained provisioning for scientific workflow ensembles in iaas clouds,” in *SC '12*, 2012, pp. 22:1–22:11.
- [13] J. Schad, J. Dittrich, and J.-A. Quiané-Ruiz, “Runtime measurements in the cloud: observing, analyzing, and reducing variance,” *Proc. VLDB Endow.*, vol. 3, no. 1-2, pp. 460–471, Sep. 2010.
- [14] A. Iosup, S. Ostermann, N. Yigitbasi, R. Prodan, T. Fahringer, and D. Epema, “Performance analysis of cloud computing services for many-tasks scientific computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 6, pp. 931–945, Jun. 2011.
- [15] H. Wang, Q. Jing, R. Chen, B. He, Z. Qian, and L. Zhou, “Distributed systems meet economics: pricing in the cloud,” in *HotCloud '10*, 2010, pp. 6–6.
- [16] S. K. Garg, R. Buyya, and H. J. Siegel, “Time and cost trade-off management for scheduling parallel applications on utility grids,” *Future Gener. Comput. Syst.*, vol. 26, October 2010.
- [17] R. Sakellariou, H. Zhao, E. Tsiakkouri, and M. D. Dikaiakos, “Scheduling workflows with budget constraints,” in *CoreGRID '07*, 2007.
- [18] A. M. Oprescu and T. Kielmann, “Bag-of-tasks scheduling under budget constraints,” in *CloudCom '10*, 2010, pp. 351–359.
- [19] S. Yi, A. Andrzejak, and D. Kondo, “Monetary cost-aware checkpointing and migration on amazon cloud spot instances,” *IEEE Trans. on Services Comput.*, 2011.
- [20] A. Andrzejak, D. Kondo, and S. Yi, “Decision model for cloud computing under sla constraints,” in *MASCOTS '10*, 2010, pp. 257–266.
- [21] N. Chohan, C. Castillo, M. Spreitzer, M. Steinder, A. Tantawi, and C. Krintz, “See spot run: using spot instances for mapreduce workflows,” in *HotCloud '10*, 2010, pp. 7–7.
- [22] M. Mattess, C. Vecchiola, and R. Buyya, “Managing peak loads by leasing cloud infrastructure services from a spot market,” in *HPCC '10*, 2010, pp. 180–188.
- [23] M. Mazzucco and M. Dumas, “Achieving performance and availability guarantees with spot instances,” in *HPCC '11*, 2011, pp. 296–303.
- [24] W. Voorsluys and R. Buyya, “Reliable provisioning of spot instances for compute-intensive applications,” pp. 542–549, 2012.
- [25] A. E. availability, <http://aws.amazon.com/ec2-sla/>.
- [26] O. A. Ben-Yehuda, M. Ben-Yehuda, A. Schuster, and D. Tsafir, “Deconstructing amazon ec2 spot instance pricing,” in *CloudCom '11*, 2011, pp. 304–311.
- [27] B. Javadi, R. Thulasiram, and R. Buyya, “Statistical modeling of spot instance prices in public cloud environments,” in *UCC '11*, 2011.
- [28] Q. Zhu and G. Agrawal, “Resource provisioning with budget constraints for adaptive applications in cloud environments,” in *HPDC '10*, 2010, pp. 304–307.
- [29] B. Bouterse and H. Perros, “Scheduling cloud capacity for time-varying customer demand,” in *CloudNet '12*, 2012.
- [30] Q. Zhang, L. Cherkasova, and E. Smirni, “A regression-based analytic model for dynamic resource provisioning of multi-tier applications,” in *ICAC '07*, 2007.
- [31] U. Sharma, P. Shenoy, S. Sahu, and A. Shaikh, “A cost-aware elasticity provisioning system for the cloud,” in *ICDCS '11*, 2011.
- [32] M. Malawski, G. Juve, E. Deelman, and J. Nabrzyski, “Cost- and deadline-constrained provisioning for scientific workflow ensembles in iaas clouds,” in *SC '12*, 2012.
- [33] R. N. Calheiros and R. Buyya, “Meeting deadlines of scientific workflows in public clouds with tasks replication,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 99, no. PrePrints, p. 1, 2013.
- [34] N. Roy, A. Dubey, and A. Gokhale, “Efficient autoscaling in the cloud using predictive models for workload forecasting,” in *CLOUD '11*, 2011, pp. 500–507.
- [35] J. Yang, J. Qiu, and Y. Li, “A profile-based approach to just-in-time scalability for cloud applications,” in *CLOUD '09*, 2009, pp. 9–16.
- [36] S. Ostermann and R. Prodan, “Impact of variable priced cloud resources on scientific workflow scheduling,” in *Euro-Par '12*, 2012, pp. 350–362.
- [37] B. Javadi, R. K. Thulasiram, and R. Buyya, “Characterizing spot price dynamics in public cloud environments,” *Future Gener. Comput. Syst.*, vol. 29, no. 4, pp. 988–999, 2013.
- [38] M. S. Rehman and M. F. Sakr, “Initial findings for provisioning variation in cloud computing,” in *CloudCom '10*, 2010, pp. 473–479.
- [39] S. Suakanto, S. H. Supangkat, Suhardi, and R. Saragih, “Performance measurement of cloud computing services,” *CoRR*, vol. abs/1205.1622, 2012.
- [40] G. Wang and T. S. E. Ng, “The impact of virtualization on network performance of amazon ec2 data center,” in *INFOCOM '10*, 2010, pp. 1163–1171.
- [41] X. Pu, L. Liu, Y. Mei, S. Sivathanu, Y. Koh, and C. Pu, “Understanding performance interference of i/o workload in virtualized cloud environments,” in *CLOUD '10*, 2010, pp. 51–58.

- [42] X. Pu, L. Liu, Y. Mei, S. Sivathanu, Y. Koh, C. Pu, Y. Cao, and L. Liu, "Who is your neighbor: Net i/o performance interference in virtualized clouds," *IEEE Trans. Services Comput.*, vol. 99, 2012.
- [43] H. Ballani, P. Costa, T. Karagiannis, and A. Rowstron, "Towards predictable datacenter networks," in *Proc. of the ACM SIGCOMM 2011 conference*, ser. SIGCOMM '11, 2011, pp. 242–253.
- [44] M. Hovestadt, O. Kao, A. Kliem, and D. Warneke, "Evaluating adaptive compression to mitigate the effects of shared I/O in clouds," in *IPDPSW '11*, May 2011, pp. 1042–1051.
- [45] X. Lin, Y. Mao, F. Li, and R. Ricci, "Towards fair sharing of block storage in a multi-tenant cloud," in *HotCloud '12*, Jun. 2012.
- [46] R. C. Chiang and H. H. Huang, "Tracon: interference-aware scheduling for data-intensive applications in virtualized environments," in *SC '11*, 2011, pp. 47:1–47:12.
- [47] "Iperf," <http://iperf.sourceforge.net>.
- [48] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," in *Future Gener. Comput. Syst.*, 2013, pp. 682–692.
- [49] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, and K. Vahi, "Characterization of scientific workflows," in *WORKS '08*, 2008.
- [50] CondorTeam, "DAGMan", "<http://cs.wisc.edu/condor/dagman>."
- [51] M. L. et al., "Condor: A hunter of idle workstations," in *ICDCS '88*, 1988, pp. 104–111.
- [52] M. Mao and M. Humphrey, "A performance study on the vm startup time in the cloud," in *CLOUD '12*, 2012, pp. 423–430.